Global Superstore — Databricks Medallion Pipeline

Project Summary: Bronze, Silver, and Gold Layers

---

1. Project Overview

This project builds a full medallion architecture data pipeline on the Global Superstore dataset (Orders, People, and Returns source files) using Databricks Free Edition with Unity Catalog, under the workspace catalog. The pipeline progresses raw CSV data through three layers — Bronze, Silver, and Gold — each with a distinct responsibility, culminating in business-ready aggregate tables intended for consumption in Power BI.

The pipeline was designed end-to-end from the outset, including a Workflow JSON configuration, and built layer by layer with a consistent working discipline: load the data, explore its actual shape, and only then transform it — rather than assuming column names, data types, or referential integrity in advance.

2. Bronze Layer

The Bronze layer ingests the raw Orders, People, and Returns CSV files into Delta tables with minimal transformation, preserving the source data as faithfully as possible while still making it queryable.

Key decisions

- inferSchema was disabled (set to false) so that column types are not silently guessed by Spark, avoiding subtle type mismatches downstream.
- CSV parsing options for quote, escape, and multiLine handling were explicitly configured to correctly parse fields containing embedded commas, quotes, and line breaks.
- A clean_columns() function was used to standardize column names (e.g., consistent casing and underscores) across the ingested tables.
- A bronze_rejected table was created to quarantine 300 rows that had corrupted Sales values, caused by embedded-quote parsing issues in the source CSV — keeping bad rows visible and auditable rather than silently dropped or silently corrupting downstream layers.

3. Silver Layer

The Silver layer reads from Bronze (not from the original CSVs) and applies cleaning and type-casting to produce reliable, properly typed tables: silver_orders, silver_people, and silver_returns.

Key decisions

- Type casting was done using SQL try_cast throughout, rather than DataFrame .cast(). This was a deliberate workaround: Databricks Serverless throws hard errors when .cast() encounters a value it cannot convert, whereas try_cast gracefully returns null instead of failing the whole job.
- Reading from Bronze rather than raw CSV ensures Silver builds on the already-ingested, quarantine-aware Bronze tables.

4. Gold Layer

The Gold layer aggregates Silver data into three business-ready tables, each aligned to a specific analytical question, and each written as a Delta table via .write.format("delta").mode("overwrite").saveAsTable(...).

4.1 gold_sales_by_region

Grouped by Market and Region together (rather than Region alone), since the Global Superstore dataset has a genuine two-level geography hierarchy — Market (e.g., APAC, EU, US) rolls up multiple Regions (e.g., Oceania, Central Asia, North Asia). Grouping by both preserves drill-down capability for Power BI without needing a second table.

```
groupBy("Market", "Region").agg(sum("Sales"), sum("Profit"), countDistinct("Order_ID"))
```

Result: 18 Market/Region combinations, including all US regions (East, West, Central, South) and international regions across APAC, EMEA, EU, LATAM, Canada, and Africa. Figures were sanity-checked for proportionality between order count and totals before saving.

4.2 gold_profit_by_category

Grouped by Category and Sub_Category, with an added Profit_Margin_Pct column (Total_Profit ÷ Total_Sales × 100, rounded to 2 decimal places) to surface profitability relative to sales volume, not just raw profit.

```
groupBy("Category", "Sub_Category").agg(sum("Sales"), sum("Profit"),
countDistinct("Order_ID"))   .withColumn("Profit_Margin_Pct", round(Total_Profit /
Total_Sales * 100, 2))
```

Result: 17 Sub-Category rows. Tables (Furniture) was the sole loss-making sub-category at −8.46% margin — a known pattern in this dataset due to high shipping and discount costs on furniture. Paper had the strongest margin at 24.24%. Copiers and Phones generated the most raw profit within Technology.

4.3 gold_return_rates

This table required joining silver_orders to silver_returns, since silver_returns only carries Order_ID, Returned, and Market — not Category or Region. This join surfaced the most significant data-quality finding in the project.

Data quality finding: non-unique Order_ID

A duplicate-key check on silver_returns (grouping by Order_ID and filtering for count > 1) revealed one Order_ID appearing twice, with different Market values (LATAM and United States) but both marked Returned = Yes. Investigating further confirmed this was not a data entry duplicate but a genuine ID collision: Order_IDs in the Global Superstore dataset are not guaranteed globally unique, since they were generated independently per market.

A follow-up check on silver_orders confirmed this was systemic, not an isolated case: 37 distinct Order_IDs span more than one Market across the order line items.

```
silver_orders.groupBy("Order_ID").agg(countDistinct("Market")).filter("distinct_markets >
1").count()  // returned 37
```

The fix: join silver_orders to silver_returns on the compound key of Order_ID and Market together, rather than Order_ID alone. This compound key was verified as unique in silver_returns (zero duplicates) before proceeding.

```
orders_with_returns = silver_orders.join(   silver_returns.select("Order_ID", "Market",
"Returned"),   on=["Order_ID", "Market"], how="left" ).withColumn("Is_Returned",
when(col("Returned") == "Yes", 1).otherwise(0))
```

Without this fix, return status could have been silently misattributed between two unrelated orders that happened to share an Order_ID across different markets — a subtle correctness bug that would not have thrown any error.

Final aggregation

```
gold_return_rates = orders_with_returns.groupBy("Category").agg(
countDistinct("Order_ID", "Market").alias("Total_Orders"),
sum("Is_Returned").alias("Returned_Orders") ).withColumn("Return_Rate_Pct",
round(Returned_Orders / Total_Orders * 100, 2))
```

Result:

Category	Total Orders	Returned Orders	Return Rate %
Office Supplies	19,023	1,310	6.89
Furniture	8,204	466	5.68
Technology	8,358	467	5.59

5. Gold Layer Summary Table

Table	Grouping	Purpose
gold_sales_by_region	Market, Region	Total sales, profit, and order count by geography, preserving Market→Region drill-down
gold_profit_by_category	Category, Sub_Category	Total sales, profit, order count, and profit margin % per product line
gold_return_rates	Category	Return rate % by category, built on a corrected Order_ID + Market join

6. Notebook Hygiene

After completing the Gold layer, the notebook was cleaned up to retain only final pipeline logic: each table's build/aggregation cell and its corresponding write (saveAsTable) cell. Exploratory and diagnostic cells — the original flawed Order_ID-only join, duplicate-key checks, and row-level inspections used to diagnose the Market collision — were removed, with the key finding preserved instead as an inline code comment on the corrected join cell.

7. Status and Next Steps

- Bronze layer: complete
- Silver layer: complete
- Gold layer: complete (all three tables built, verified, and saved as Delta tables)
- Notebook cleanup: complete for the Gold section
- Next step: connect the Gold layer tables to Power BI for dashboarding
- Also planned separately: a hands-on session on manually configuring Databricks clusters on Azure or AWS free tier, to practice a production-style workspace setup